

Compiler Design

Final project. This is a group project, and all members will receive the same score. All members must attend the demonstration day, and I will pick up a member to do the demonstration.

Create the following text file: “final.txt”

```
1.      program f2024;
2.      (* This program computes and prints the value
3.        of an expression *)
4.      var
5.        (* declare variables *)
6.        a ,    b2a ,    c, bba   : integer ;
7.      begin
8.        a          = 3 ;
9.        b2a =      14 ;
10.       c          = 5  ;
11.       print ( c ); (* display c *)
12.
13.       (* compute the value of the expression *)
14.       bba = a1 * ( b2a + 2 * c)      ;
15.       print ( “value=”,      bba  ); (* print the value of bba*)
16.
17.       end
```

Part I (10 points)

Apply the following rules to this file and copy the new version in file “final24.txt”

- Any line/s or part of a line that begins with (*) and ends with *) is/are considered as a comment line(s) (i.e. lines #2-3,5, 11,13,15), remove them all
- Any blank lines must be removed (i.e. lines #12, 16)
- Extra spaces in each line must be removed, leaving one space before and one after each token to make tokenization easier.

The “finalf24.txt” should look like this

```
1.      program f2024;
2.      var
3.      a , b2a , c , bba : integer ;
4.      begin
5.      a = 3 ;
6.      b2a = 14 ;
7.      c = 5 ;
8.      print ( c );
8.      bba = ( b2a + 2 * c ) * a ;
9.      print ( “Value=”, bba );
10.     end
```

Part II (10 points)

Translate the file “final24” to one of the following C++, C#, Java or Python languages, run the program, and display its output. For example, the translation to C++ should look like this

```
#include <iostream>
using namespace std;
int main()
{
    int a , b2a , c , bba ;
    a = 3 ;
    b2a = 14;
    c = 5 ;
    cout<< c << endl;
    bba = ( b2a + 2 * c ) * a ;
    cout<< “Value=” <<bba << endl;
    system (“pause”);
    return 0;
}
```

Output:

```
5
Value=72
```

Part III (50 points)

Use the following grammar (the part that is highlighted is the one you used in Handouts 7 and 8)

<prog>	→ program <identifier>; var <dec-list> begin <stat-list> end
<identifier>	→ <letter>{<letter> <digit>} note. This grammar is in EBNF
<dec-list>	→ <dec> : <type> ;
<dec>	→ <identifier>,<dec> <identifier> >
<type>	→ integer
<stat-list>	→ <stat> <stat> <stat-list>
<stat>	→ <write> <assign>
<write>	→ print (<str> <identifier>);
<str>	→ ”value=”, λ
<assign>	→ < identifier> = <expr>;
<expr>	→ <expr> + <term> <expr> - <term> <term>
<term>	→ <term> * <factor> <term> / <factor> <factor>
<factor>	→ < identifier> <number> (<expr>)
<number>	→ <sign><digit>{ <digit> } note: the grammar is in EBNF
<sign>	→ + - λ
<digit>	→ 0 1 2 ... 9
<letter>	→ a b c d l f

In which **program**, **var**, **begin**, **end**, **integer**, and **print** are reserved words.

Have your program (compiler) check the grammar of the program and display “No Error” if the given grammar liked the program or issue “some errors” otherwise.

Have your program detect the following errors and issue a corresponding error message:

program	is expected (if program is spelled wrong)
var	is expected (if var is spelled wrong)
begin	is expected (if begin is spelled wrong)
end	is expected (if end is spelled wrong)
integer	is expected (if integer is spelled wrong)
print	is expected(if print is spelled wrong)

unknown identifier if the variable is not defined

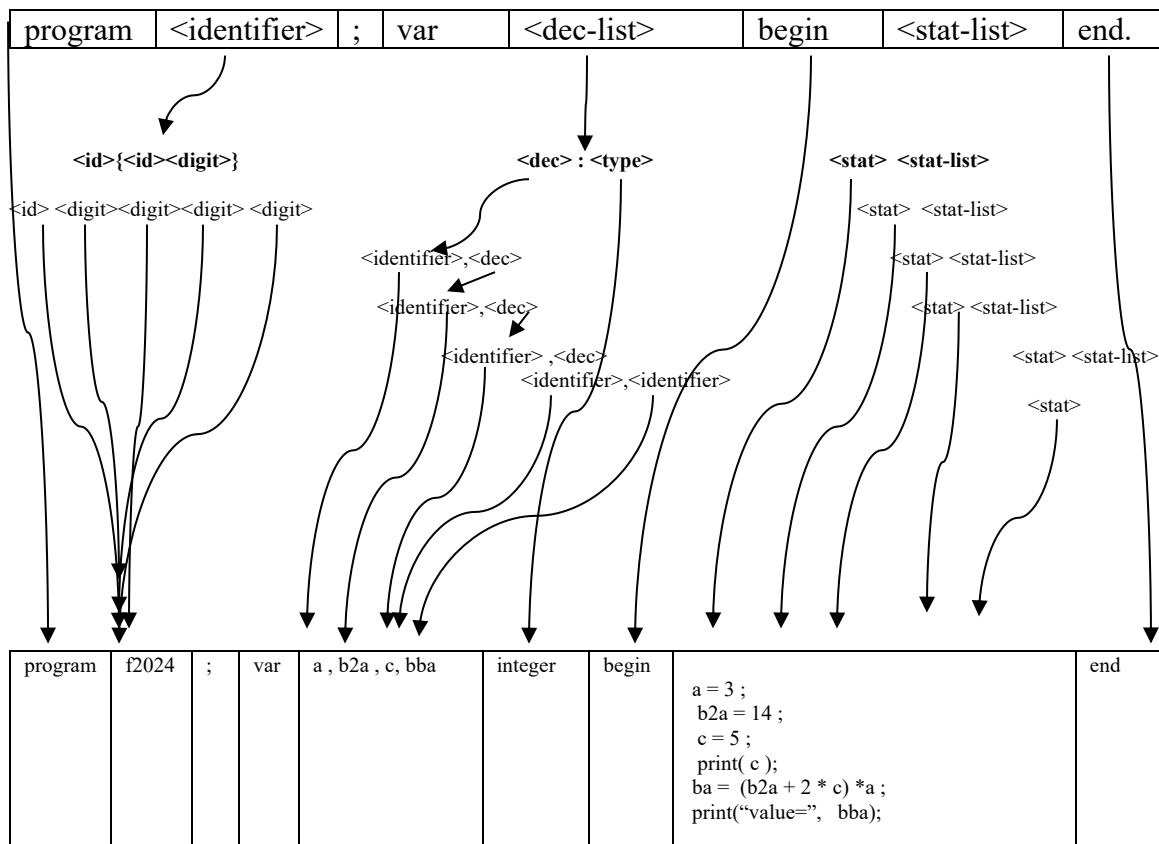
;	semicolon is missing (if the grammar required a ;)
,	comma is missing (if the grammar required a ,)
.	period is missing (if the grammar required a .)
(	The left parentheses is missing
)	The right parentheses is missing

Output: “Ready to compile” or “this is the list of errors”

Part IV (20 points)

You will receive up to 20 points if your program displays the corresponding error messages in part III

The whole program grammar consists of ONE line, and each non-terminal in the grammar will be defined over and over until it hits a terminal.



Part IV.(10 points) Turn in the following in a folder along with a flash drive that includes the whole report and a copy of the program as well. Everything must be typed (no handwriting is accepted).

1. Cover page:

Computer Science 323 Fall 2024 Final Project Group Members Method Used: Language Used:
--

2. The original program on page "1"
3. The original grammar on page "2"

4. The grammar in BNF form (remove all {, }, and |)

The original grammar in BNF after removing all EBNF grammar.	
<prog>	→ program <identifier>, var <dec-list> begin <stat-list> end
<identifier>	→ <letter> <post-identifier>
<post-identifier>	→ <letter> <post-identifier>
<pos-identifier>	→ <digit> <post-identifier>
<post-identifier>	→ λ
.	
.	
<letter>	a
<letter>	b
<letter>	c
<letter>	d
<letter>	f

5. If you are using a predictive parsing table, remove all left recursions. If you are using LR parsing table, remove all lambdas. Show the final form of the grammar in BNF

Final BNF grammar for predictive paring/LR parsing method	
.	
.	
E	→ E + T
E	→ E – T
E	→ T
.	.
L	→ d
L	→ f

6. Find the members of first and follow

Non-terminals		FIRST	FOLLOW
P	<prog>	program	\$
I	<identifier>	a b c l f	a b c d f + - / * ; , : 0 1 2
.....			

7. Show the parsing table

8. Complete copy of the program including all user's defined libraries, and a copy of the sample run